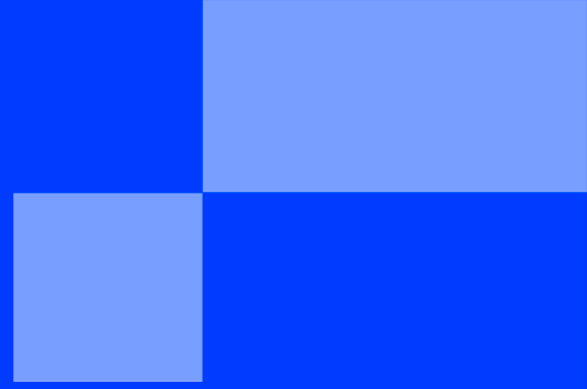
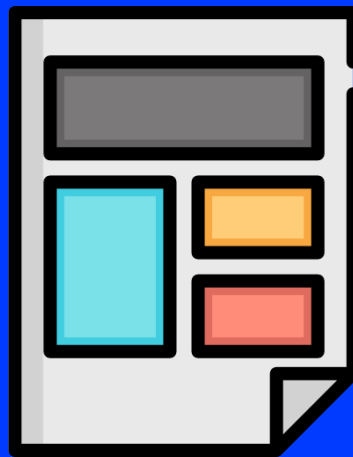




# CORELAB



< BEING A STUDENT IS EASY. LEARNING  
REQUIRES ACTUAL WORK. />



# CORELAB

## Introduction

L'objectif de ce projet est de développer un **LMS (Learning Management System)**. Un LMS est une plateforme permettant de créer, gérer, diffuser et suivre des formations à distance.

Un LMS efficace repose sur trois piliers fondamentaux qui interagissent en permanence pour optimiser les processus :

- ✓ **Administration** (inscription des utilisateurs, attribution des droits d'accès...)
- ✓ **Création et diffusion des contenus de formation** (configuration des modules, intégration de documents multimédias...)
- ✓ **Expérience d'apprentissage** (interface utilisateur intuitive, accès au catalogue des ressources...)



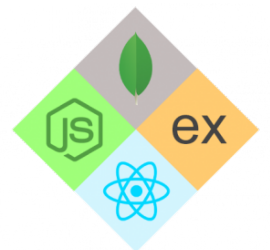
Vous avez de l'expérience avec les LMS, puisque ce sujet a été téléchargé sur l'un d'eux.

## Considérations techniques

### Stack imposée

Pour ce projet, vous devez utiliser la stack MERN :

- ✓ MongoDB
- ✓ Express.js
- ✓ React.js
- ✓ Node.js



Vous devrez créer l'intégralité du site web vous-même : **conception, développement, tests** et itérations régulières.

Mais vous ne serez pas seuls. Ces tâches seront partagées avec vos coéquipiers, ce qui signifie que **la communication, le compromis, le débat** et toutes les autres compétences relationnelles seront essentielles à la réalisation de votre objectif (probablement plus encore que la simple connaissance de MERN).



Teamwork makes the dream work.

## Contraintes liées au métier

---

Un LMS manipule des données sensibles (identifiants étudiants, résultats d'évaluations) et doit garantir la confidentialité du parcours de chacun. Votre solution doit donc respecter les exigences suivantes :

- ✓ Les mots de passe sont hachés avec **bcrypt** (cost factor  $\geq 10$ ). L'utilisation de SHA1, MD5, ou le stockage en clair entraîne une note de zéro sur l'axe Architecture & sécurité de la plateforme.
- ✓ L'authentification repose sur des tokens **JWT** avec expiration définie et vérification côté API.
- ✓ Les routes manipulant des données utilisateur ou pédagogiques sont protégées et retournent les codes HTTP appropriés (**401** pour non-authentifié, **403** pour non-autorisé).
- ✓ Les rôles **administrateur** et **étudiant** sont distingués et appliqués à toutes les routes sensibles.
- ✓ Les données entrantes (imports, formulaires, payloads d'API) sont validées avec **Zod** ou **Joi** avant traitement.
- ✓ Les variables d'environnement (secrets, URI de base de données...) sont gérées via un fichier `.env` non versionné.



Ces contraintes ne sont pas négociables : un LMS qui les ignore n'est pas un LMS livrable en production.

## Standards de qualité attendus

---

À l'issue de la piscine MERN, vous êtes attendus sur les pratiques suivantes. Elles ne relèvent pas du cahier des charges du client mais des standards professionnels que vous consolidez à travers ce projet :

- ✓ **Architecture** : séparation claire `client/` et `server/`, scripts `package.json` fonctionnels (`start`, `dev`, `build`) côté client et serveur, démarrage sans erreur depuis un environnement vierge (`npm install && npm start`).
- ✓ **Modélisation** : schémas Mongoose modélisant a minima les entités `Course` (cours / formation), `Lesson` (leçon individuelle) et `Quiz` (évaluation), en cohérence avec le vocabulaire standardisé du domaine LMS. Vous pouvez ajouter d'autres entités selon les besoins identifiés (notifications, cohortes, tentatives...).
- ✓ **Front-end** : composants React fonctionnels (hooks uniquement, pas de classes), navigation via **React Router v6**, interface **responsive**.
- ✓ **Tests** : les routes API critiques sont couvertes par des tests automatisés (**Jest** ou **Vitest**).
- ✓ **Travail en équipe** : développement via des branches par fonctionnalité et des **Pull Requests** relues par au moins un coéquipier. Aucun push direct sur `main`.



Pour faciliter le test par l'équipe pédagogique, l'API doit être accessible sur le port **4242** et le client sur le port **3000**.

# Cahier des charges

Parlons de nos trois piliers. Les fonctionnalités qui y sont associées seront formulées sous forme d'**User Stories**.

**Elles ne sont pas classées** dans un ordre particulier. C'est à vous de déterminer la priorité à attribuer à chacune.

Cependant, elles peuvent-être considérées comme validées par votre Product Owner comme les fonctionnalités à implémenter pour le premier sprint de ce projet.



Ces User Stories sont aussi fournies sous forme de CSV afin que vous puissiez aisément les importer dans un logiciel tel que Trello, Notion...



Est-ce que toutes les User Stories sont claires ? Valident-elles toutes les critères pour remplir leur rôle ? Les comprenez-vous tous de la même manière ?

## Administration

- ✓ En tant qu'administrateur, je souhaite importer une liste d'utilisateurs afin de permettre aux étudiants d'accéder aux cours.
- ✓ En tant qu'administrateur, je souhaite également choisir quel utilisateur a accès à quel cours afin de créer des classes ou cohortes d'étudiants.
- ✓ En tant qu'étudiant, je souhaite pouvoir choisir mon mot de passe lors de ma première connexion afin d'être sûr qu'un autre étudiant ne puisse pas avoir accès à mes résultats.

## Création et diffusion des contenus de formation

- ✓ En tant qu'administrateur, je souhaite pouvoir importer une leçon écrite en HTML afin de disposer d'un format flexible avec des images et des vidéos.
- ✓ En tant qu'administrateur, je souhaite pouvoir modifier une leçon existante.
- ✓ En tant qu'administrateur, je souhaite pouvoir importer un test QCM depuis un fichier structuré (JSON, CSV ou format texte documenté de mon choix), contenant pour chaque question son énoncé, ses réponses possibles et la ou les bonnes réponses, afin d'évaluer les étudiants.
- ✓ En tant qu'administrateur, je souhaite pouvoir choisir la date à laquelle une leçon sera mise à disposition d'un groupe d'étudiants afin de leur créer des parcours personnalisés.
- ✓ En tant qu'administrateur, je souhaite définir un seuil de réussite pour chaque examen afin que les

étudiants puissent suivre un parcours d'apprentissage structuré.

## Expérience d'apprentissage

- ✓ En tant qu'étudiant, je souhaite voir ma progression dans un parcours afin de planifier mes séances de progression.
- ✓ En tant qu'étudiant, je souhaite voir si j'ai échoué à une étape afin de rattraper l'exercice.
- ✓ En tant qu'étudiant, je souhaite accéder au contenu des leçons pour étudier.
- ✓ En tant qu'étudiant, je souhaite pouvoir passer un examen QCM pour mesurer mes progrès.
- ✓ En tant qu'administrateur, je souhaite consulter les notes de mes étudiants afin de les aider sur les points qu'ils ne maîtrisent pas.
- ✓ En tant qu'étudiant, je souhaite être notifié automatiquement lorsqu'une nouvelle leçon devient disponible (à la date planifiée par l'administrateur) afin de ne pas la manquer.



Le canal de notification (in-app, email, ou autre) est laissé au choix de votre équipe et doit être justifié lors de la défense.

...

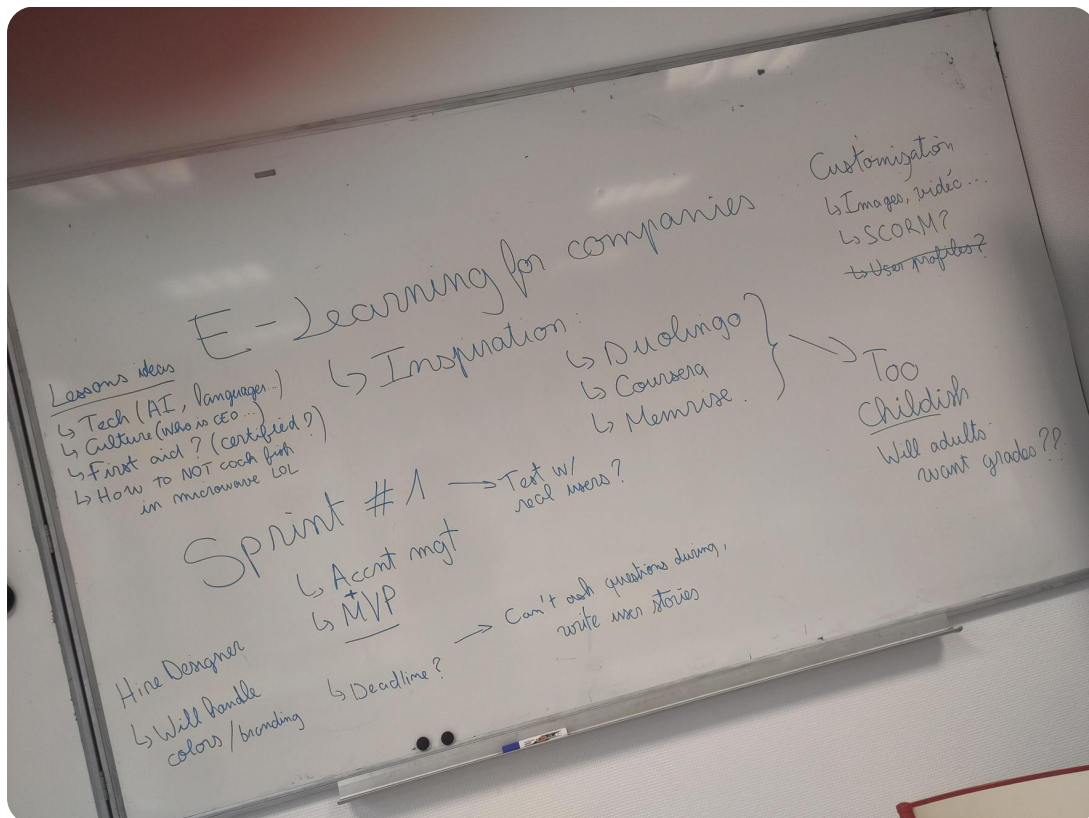
Vous avez eu une réunion avec le client, vous avez beaucoup parlé, noté beaucoup de choses au tableau, pris une photo et oublié la majeure partie de la réunion. La photo que vous avez prise est jointe à ce sujet. Vous y trouverez peut-être des informations intéressantes ou des pistes de réflexion.



Le tableau photographié contient des pistes que le client n'a pas formalisées en user stories. À vous de juger lesquelles ajouter à votre backlog, et de justifier vos choix lors de la défense.



Soyez-prêts à formuler de nouvelles user stories si vous souhaitez intégrer des fonctionnalités supplémentaires à votre projet.



## Livrables

À l'issue du projet, votre groupe rend les éléments suivants.

### Dépôt de code

Votre dépôt Git doit contenir au minimum :

- ✓ Les répertoires `client/` et `server/` avec leur code source
- ✓ Un fichier `README.md` à la racine permettant à un évaluateur de lancer le projet depuis un environnement vierge (prérequis, installation, variables d'environnement, commandes de démarrage)
- ✓ Un fichier `.env.example` listant les variables d'environnement attendues (sans valeurs sensibles)

### Suivi de projet

L'URL de votre outil de suivi (Notion, Jira, Trello, GitHub Projects...), accessible en lecture par l'équipe pédagogique, contenant :

- ✓ Le backlog complet, priorisé et estimé
- ✓ Le découpage en sprints avec leur contenu
- ✓ Un compte rendu de rétrospective de fin de projet (1 page maximum)

## Défense

---

Une présentation orale de votre projet (**30 minutes, questions/réponses incluses**), couvrant a minima :

- ✓ Une démonstration fonctionnelle de l'application
- ✓ Les choix techniques majeurs et leur justification
- ✓ L'organisation de l'équipe et la rétrospective du projet
- ✓ Les fonctionnalités ajoutées au backlog initial et leur justification
- ✓ La présentation du wireframe initial réalisé en bootstrap et l'analyse de ses écarts avec la version finale



L'absence de l'un de ces livrables au moment de la défense rend certains axes du barème non évaluable. Anticipez leur préparation tout au long du projet, pas la veille de la soutenance.

v 1.1

{EPITECH}